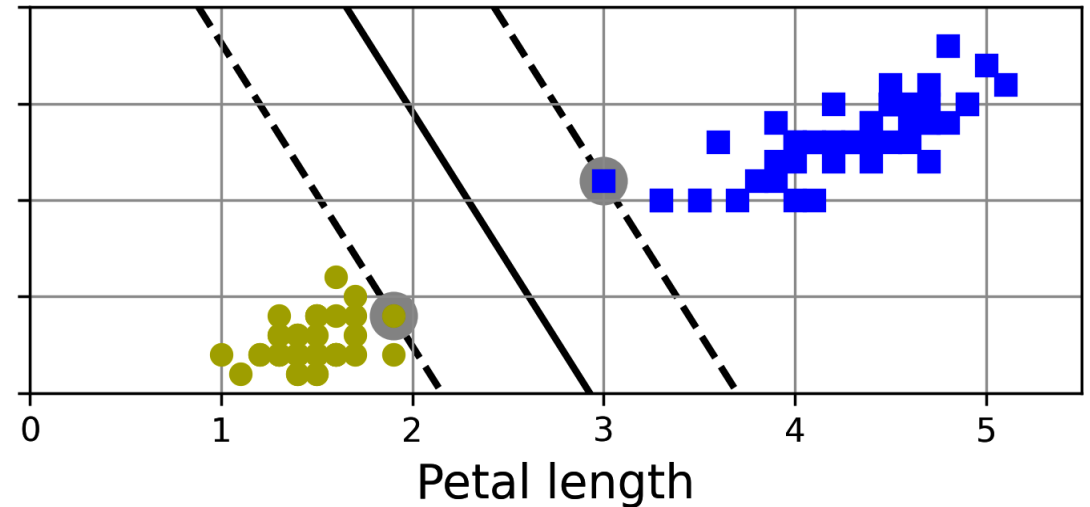
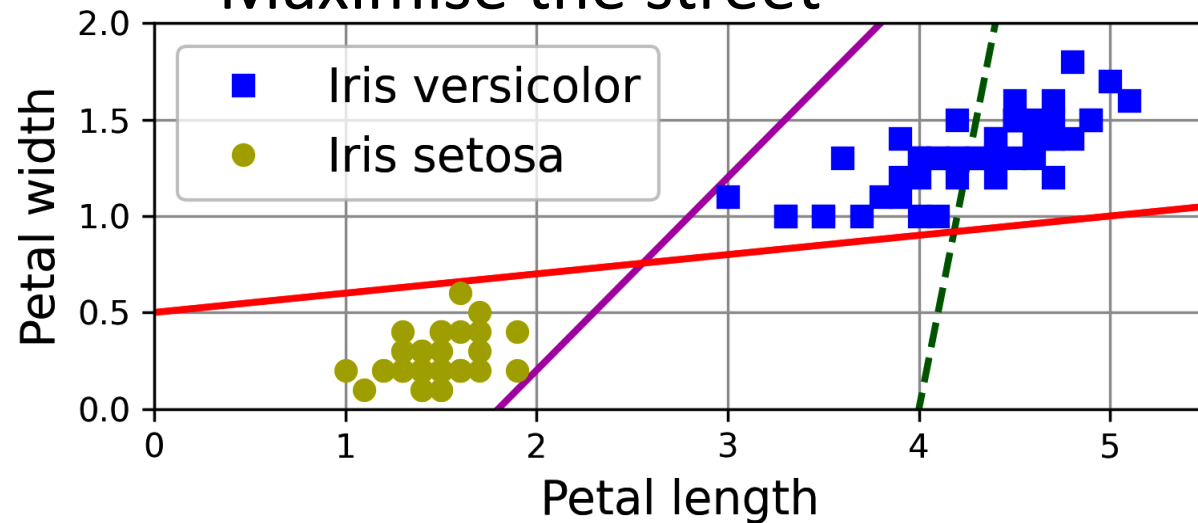


Topic 8

SVM

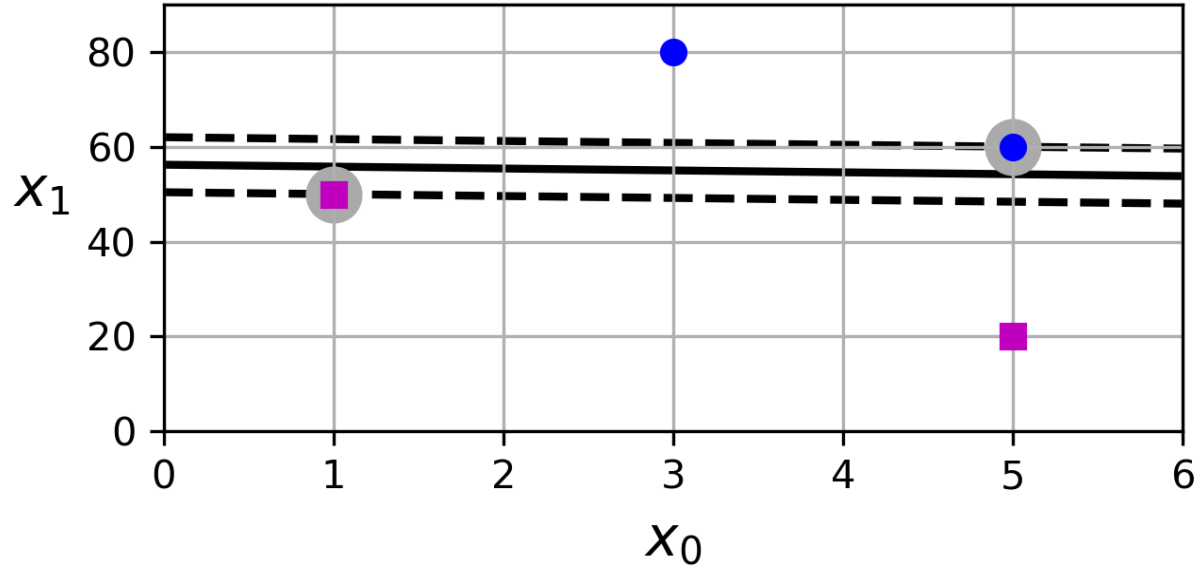
Linear SVM Classification

- Large Margin Classifier
- Support vectors definition
- Maximise the street

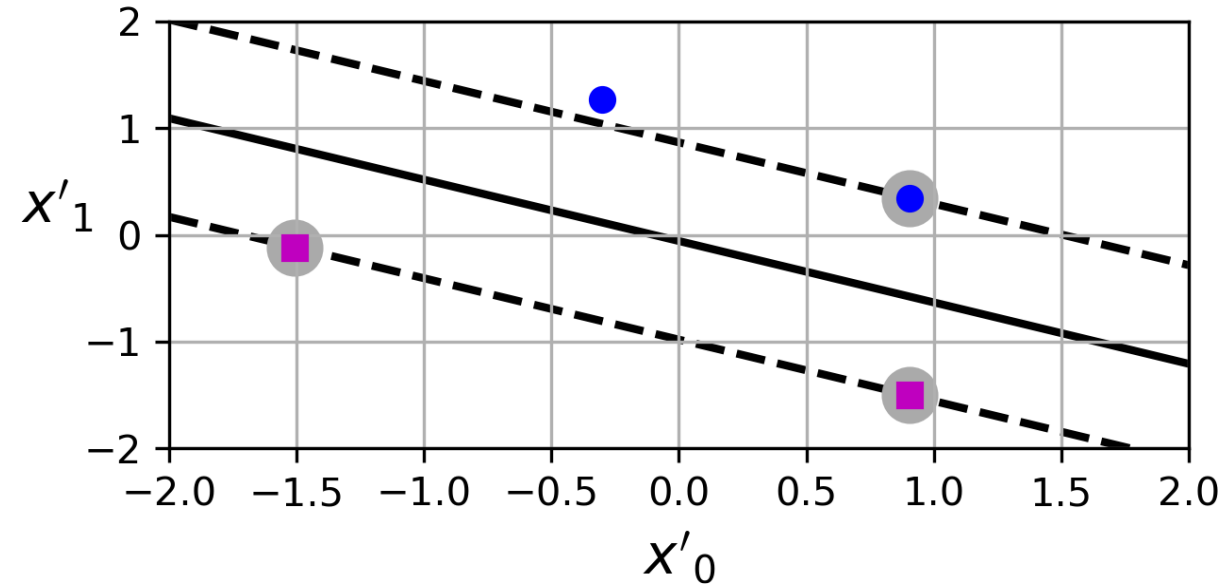


Sensitivity to Feature Scaling

Unscaled

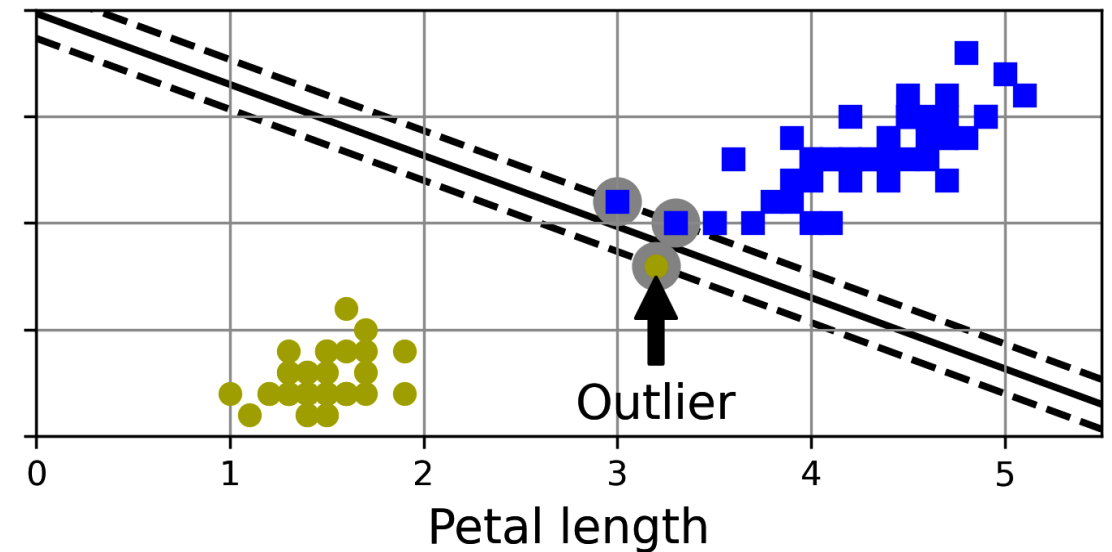
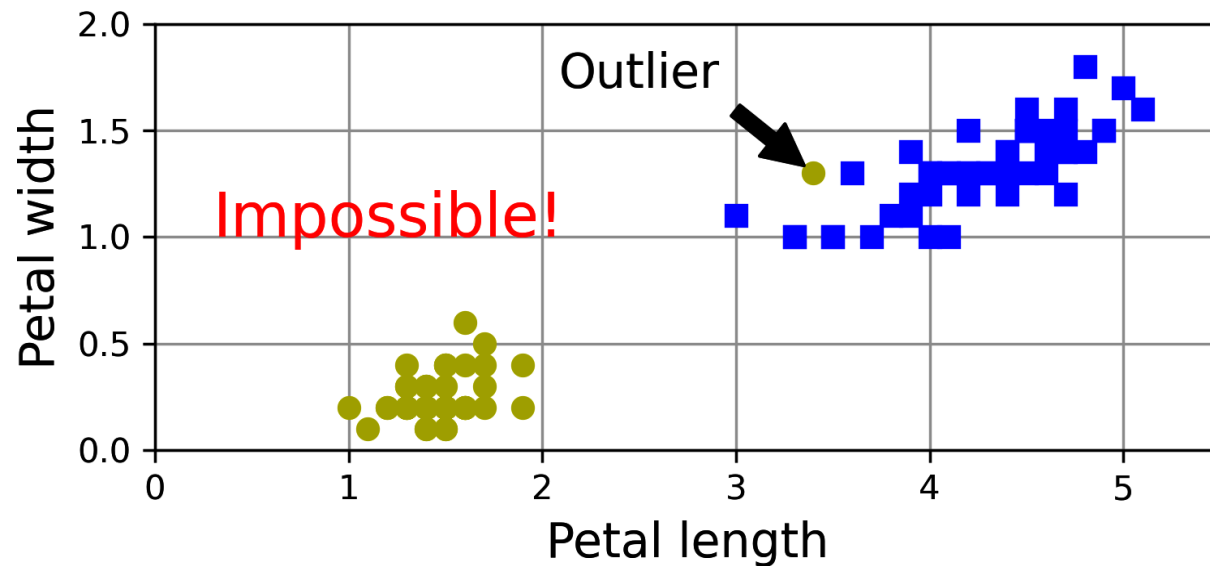


Scaled



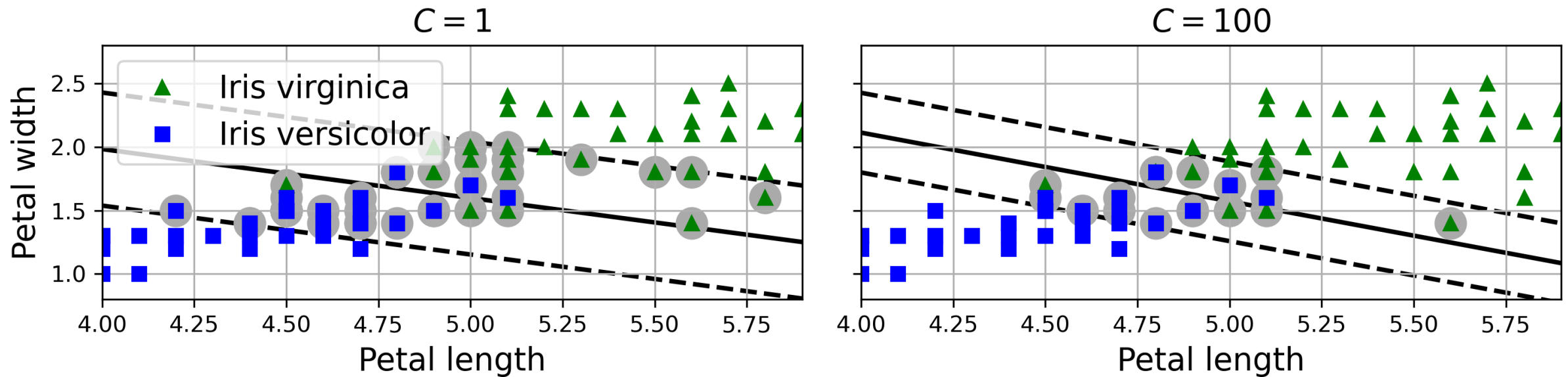
Hard and Soft Margins

- If all instances must be off the street and on the correct side, this is called *hard margin classification*
- The objective is to find a good balance between keeping the street as large as possible and limiting the *margin violations* (i.e., instances that end up in the middle of the street or even on the wrong side). This is called *soft margin classification*.



Regularization Hyperparameter

- Regularization Hyperparameter C
- If you set it to a low value, then you end up with the model on the left of.
- With a high value, you get the model on the right. As you can see, reducing C makes the street larger, but it also leads to more margin violations. In other words, reducing C results in more instances supporting the street, so there's less risk of overfitting. But if you reduce it too much, then the model ends up underfitting



```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # Iris virginica

svm_clf = make_pipeline(StandardScaler(),
                        LinearSVC(C=1, random_state=42))
svm_clf.fit(X, y)
```

Then, as usual, you can use the model to make predictions:

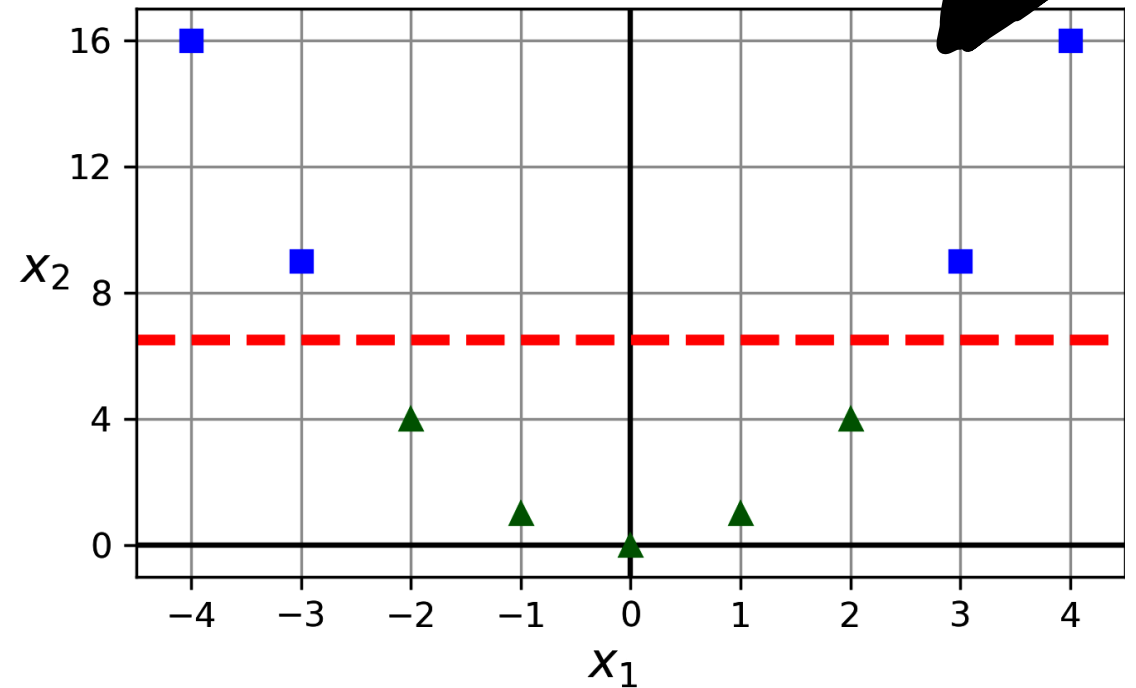
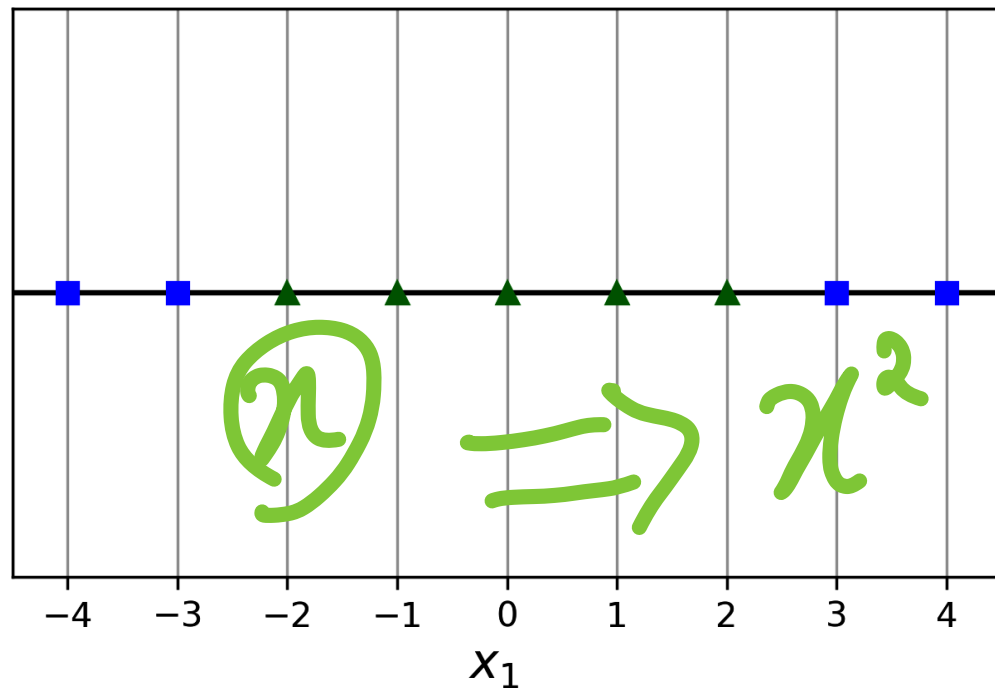
```
>>> X_new = [[5.5, 1.7], [5.0, 1.5]]
>>> svm_clf.predict(X_new)
array([ True, False])
```

The first plant is classified as an *Iris virginica*, while the second is not. Let's look at the scores that the SVM used to make these predictions. These measure the signed distance between each instance and the decision boundary:

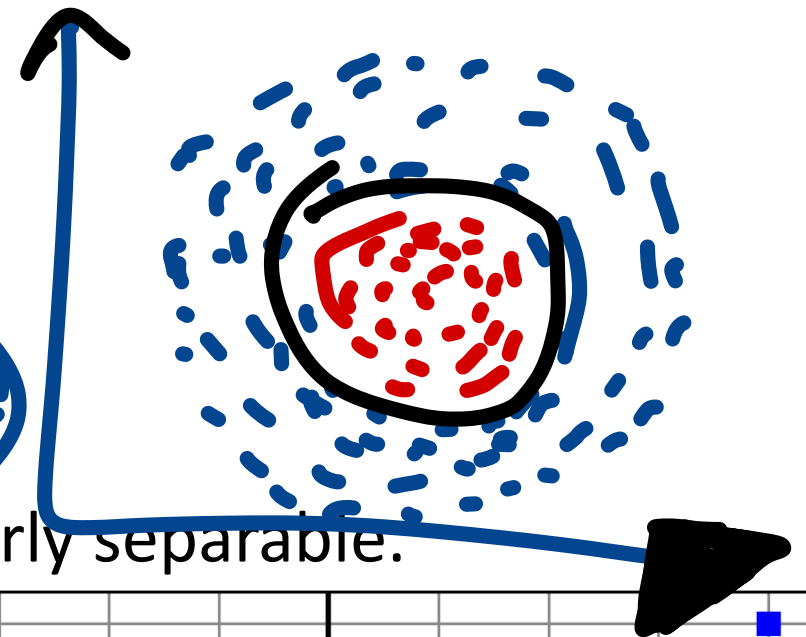
```
>>> svm_clf.decision_function(X_new)
array([ 0.66163411, -0.22036063])
```

Nonlinear SVM Classification

- Linear data set not separable.
- Adding a feature $x_2 = (x_1)^2$ makes data linearly separable.



2D



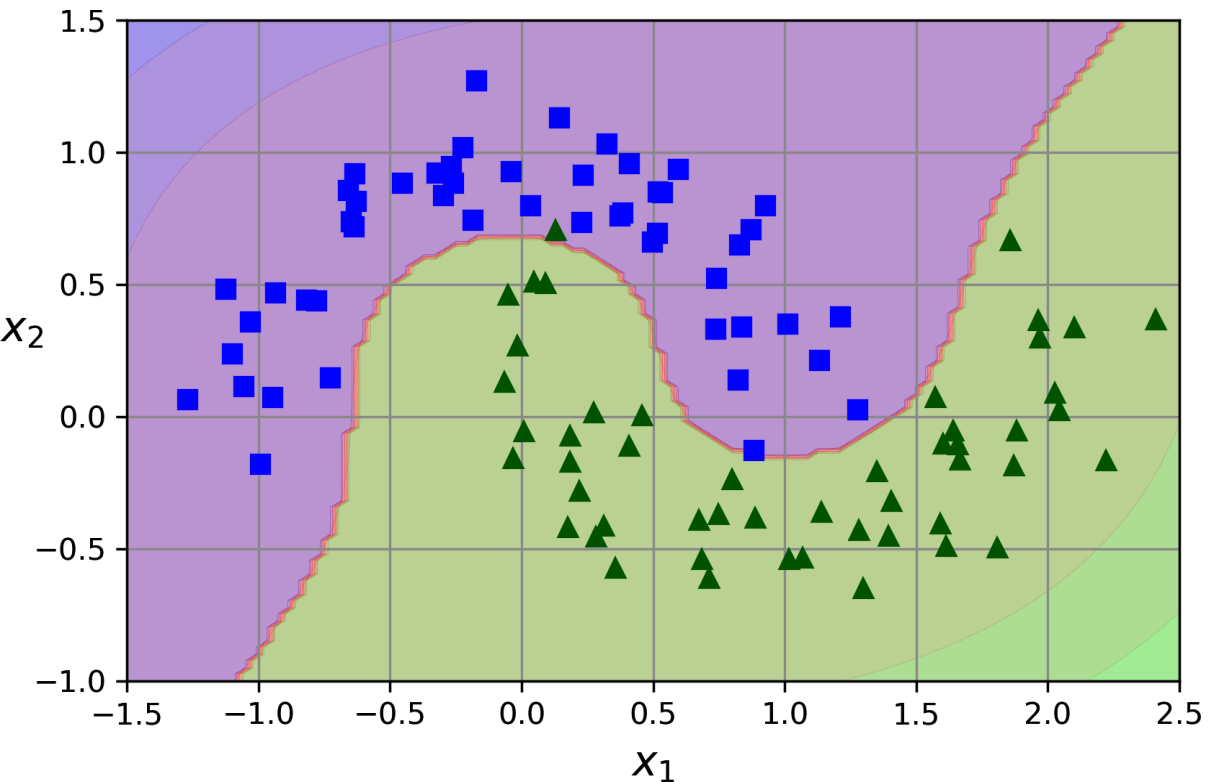
1D

Polynomial Feature Transformation

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures

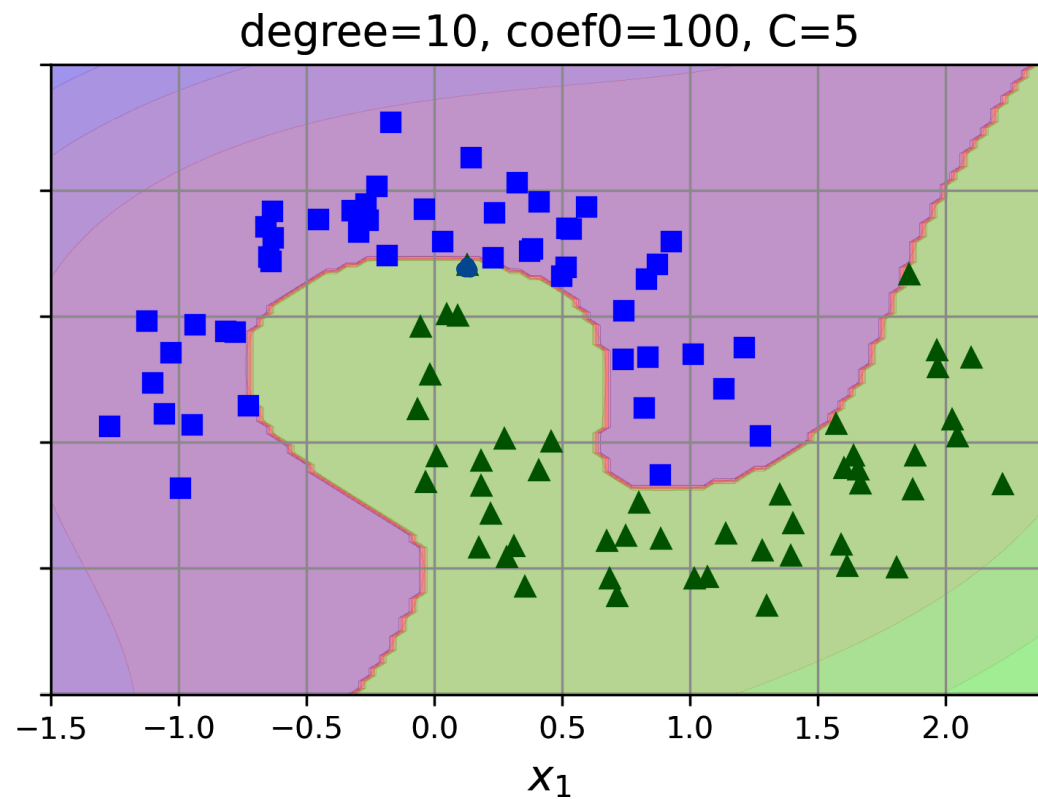
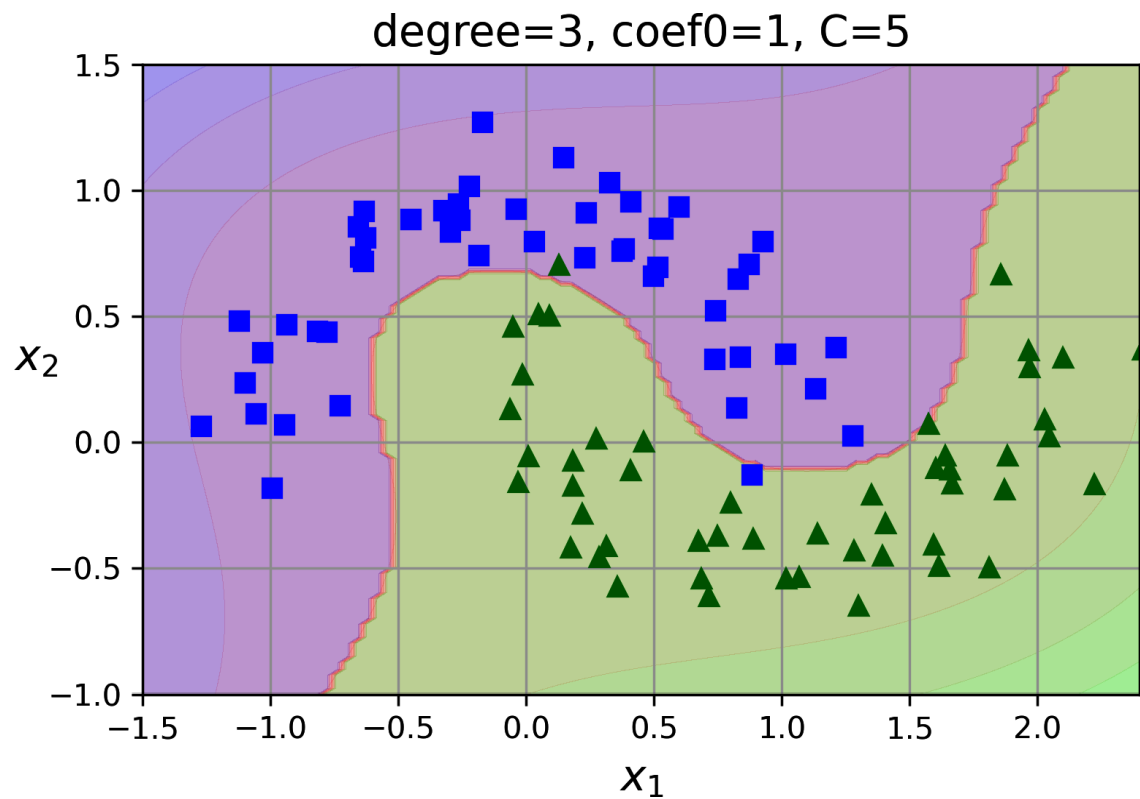
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
)
polynomial_svm_clf.fit(X, y)
```



```
from sklearn.svm import SVC

poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                     SVC(kernel="poly", degree=3, coef0=1, C=5))
poly_kernel_svm_clf.fit(X, y)
```



Kernel Trick - Intro

- Map the points to a 9 dimensional space
- Quite computationally complex
- Need to compute the final result

- Kernel function $k(x,y)$ – instead of doing the complicated computations in the 9 dimensional space, we reach the same result by calculating the dot product of x -transpose and y .

$$\mathbf{x} = (x_1, x_2, x_3)^T$$

$$\mathbf{y} = (y_1, y_2, y_3)^T$$

$$\phi(\mathbf{x}) = (x_1^2, x_1x_2, x_1x_3, x_2x_1, x_2^2, x_2x_3, x_3x_1, x_3x_2, x_3^2)^T$$

$$\phi(\mathbf{y}) = (y_1^2, y_1y_2, y_1y_3, y_2y_1, y_2^2, y_2y_3, y_3y_1, y_3y_2, y_3^2)^T$$

$$\phi(\mathbf{x})^T \phi(\mathbf{y}) = \sum_{i,j=1}^3 x_i x_j y_i y_j$$

$$\begin{aligned} k(\mathbf{x}, \mathbf{y}) &= (\mathbf{x}^T \mathbf{y})^2 \\ &= (x_1y_1 + x_2y_2 + x_3y_3)^2 \\ &= \sum_{i,j=1}^3 x_i x_j y_i y_j \end{aligned}$$

(Linear)

$$K(a, b) = \underline{a^T b} + c$$

(Polynomial)

$$K(a, b) = (\gamma \underline{a^T b} + r)^d$$

Gaussian:

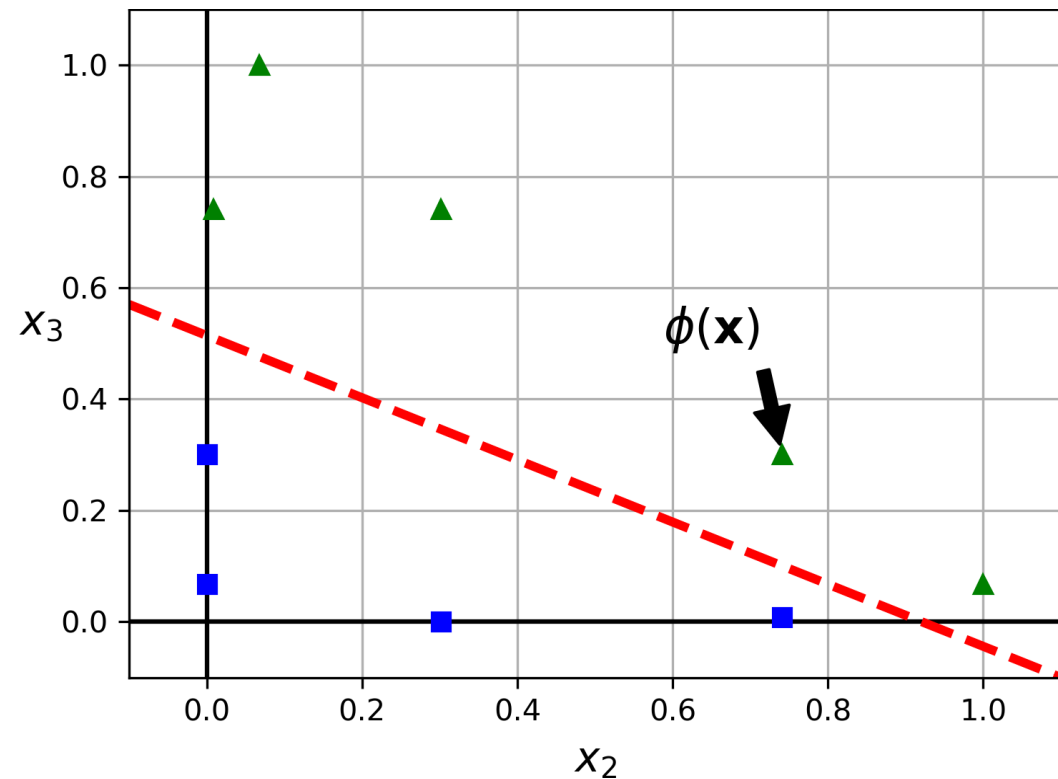
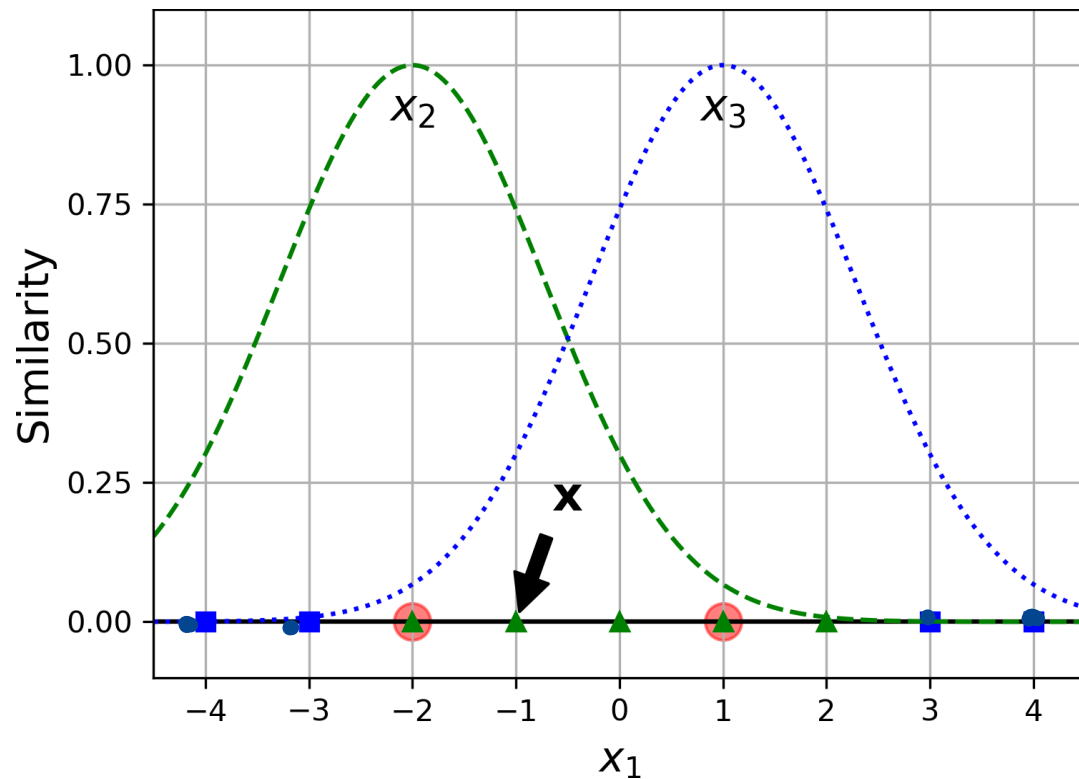
$$\underline{K(a, b) = \exp(-\gamma \|a - b\|^2)}$$

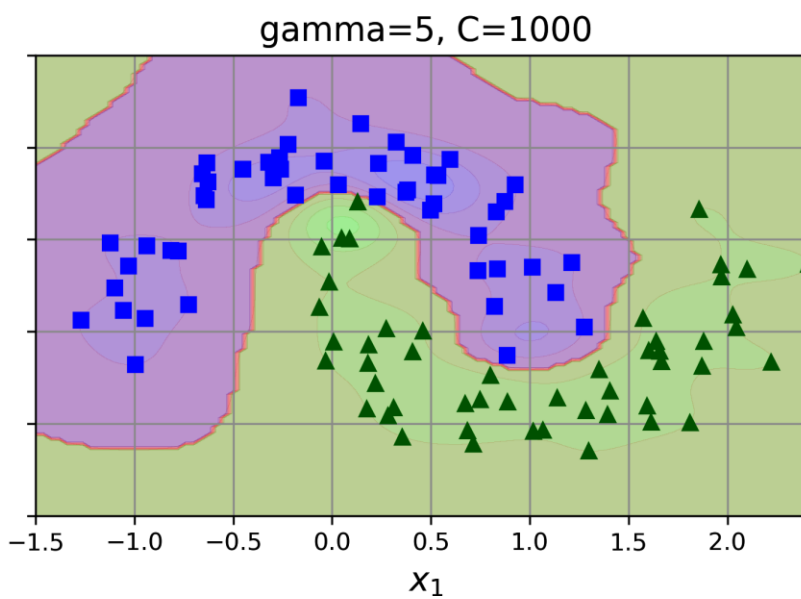
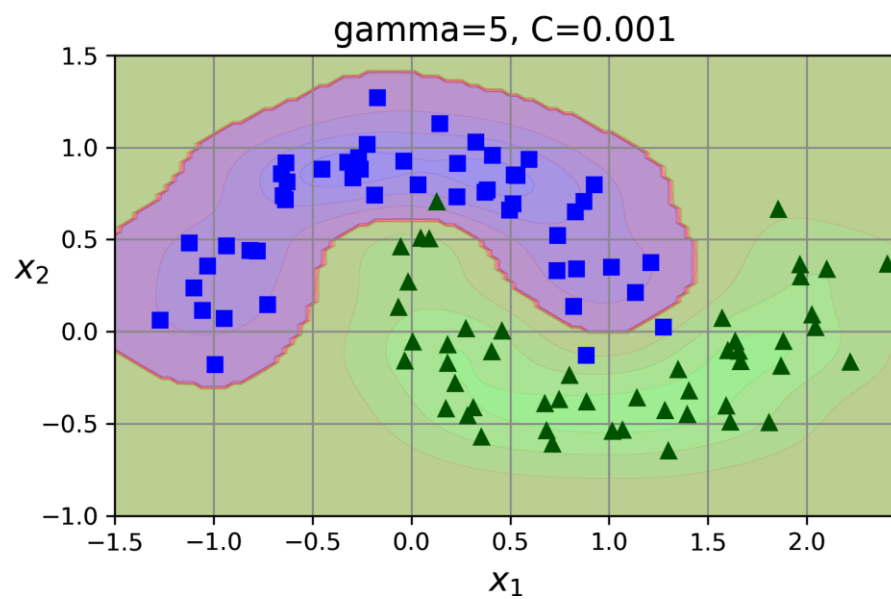
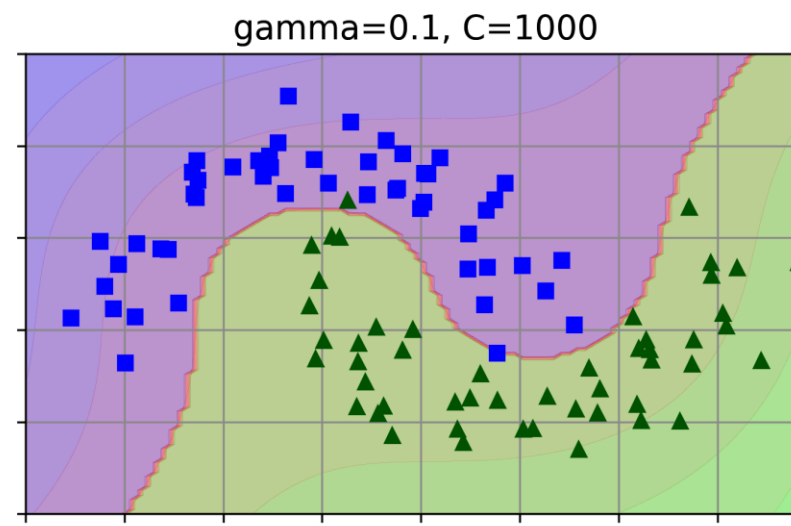
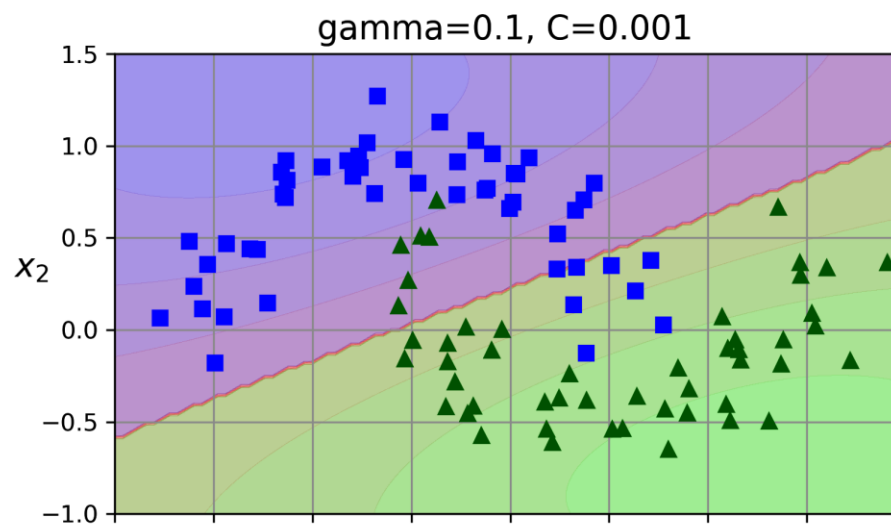
Sigmoid:

$$\tanh(\gamma a^T b + r)$$

Similarity Features

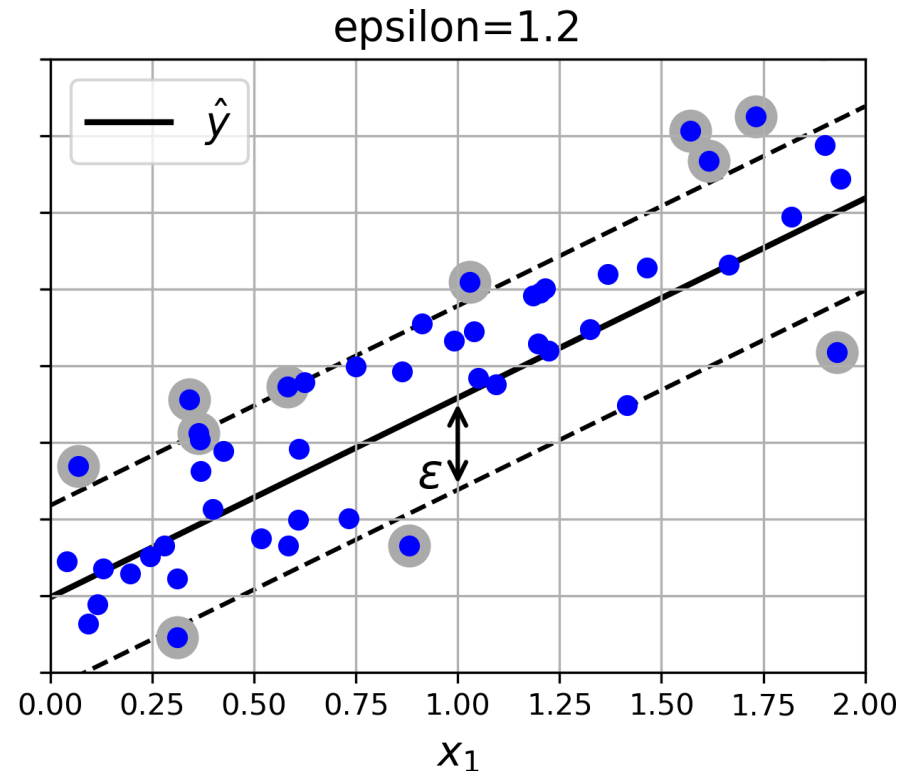
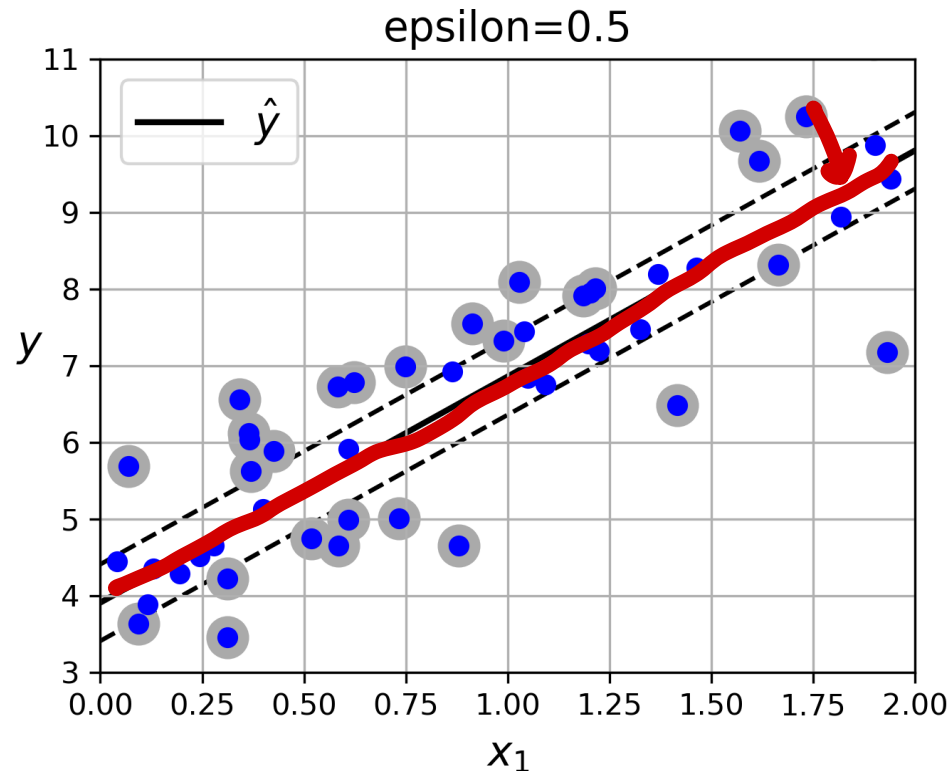
- Transformation using the Gaussian Radial Basis Function



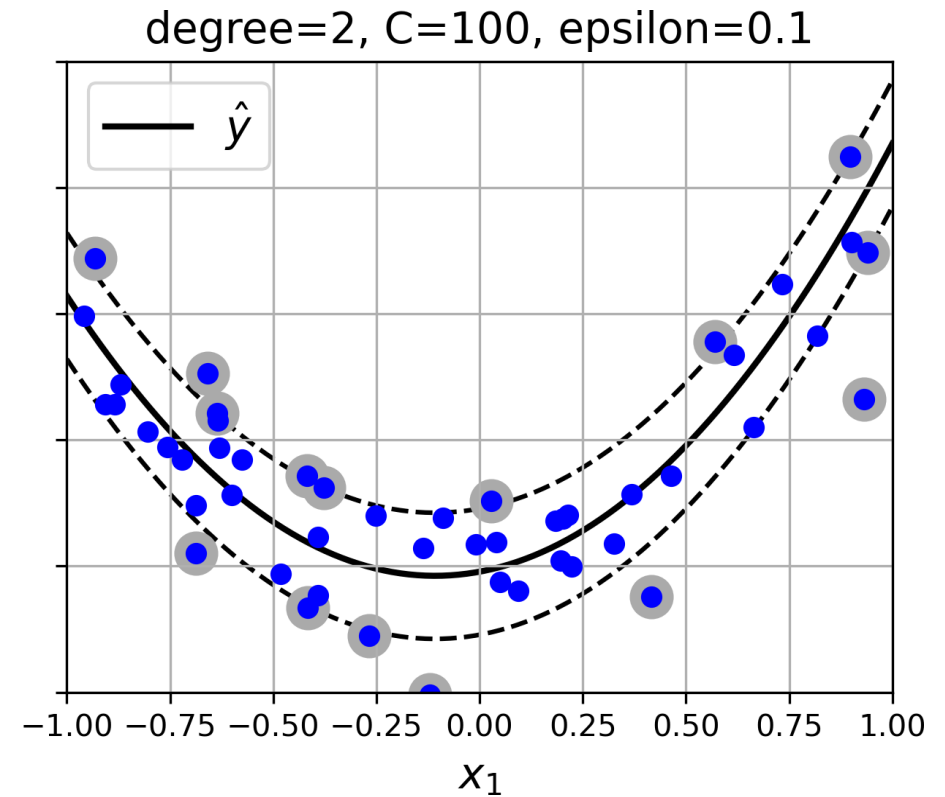
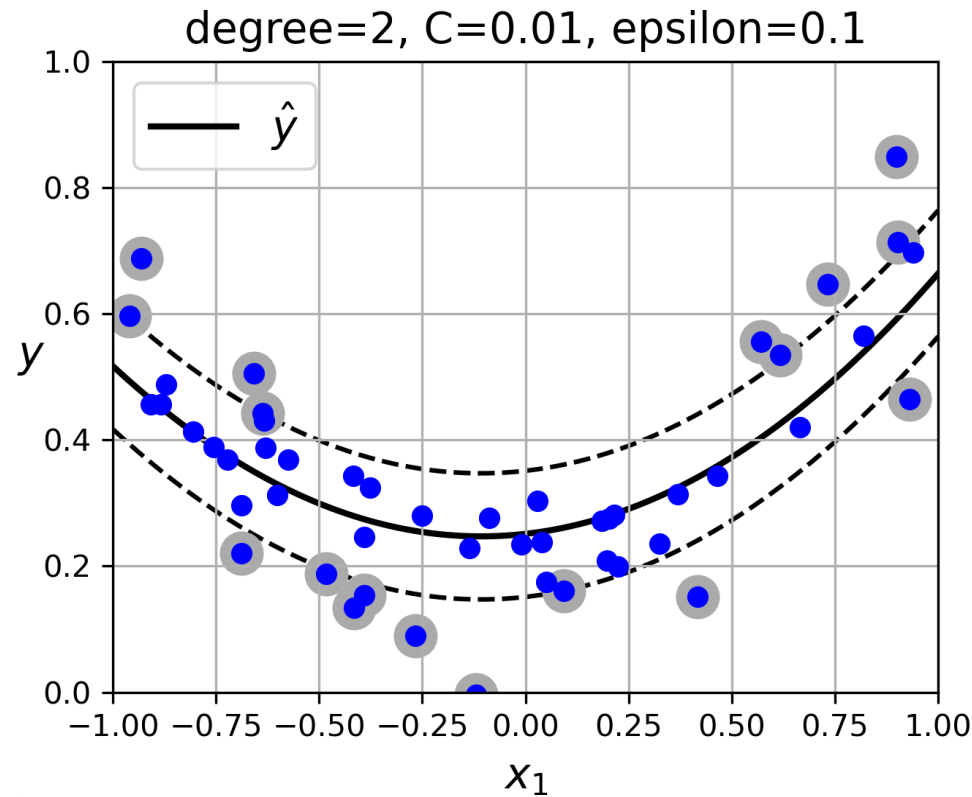


SVM Regression

- Tries to put instances on the street instead of away from the street.
- The width of the street has hyperparameter



Non-Linear Regression



```
from sklearn.svm import SVR
```

```
X, y = [...] # a quadratic dataset
```

```
svm_poly_reg = make_pipeline(StandardScaler(),  
                             SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))
```

```
svm_poly_reg.fit(X, y)
```

